

10.1 Why not invert the cross-product matrix

Chapter 5 found the least squares estimate by setting a gradient to zero, which gave $\hat{\beta} = (\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top \mathbf{y}$ when $\mathbf{X}$ has full column rank (Theorem 5.4.1). Read as a recipe, the formula says: form the $p \times p$ matrix $\mathbf{X}^\top \mathbf{X}$, invert it, and multiply. Section 6.10 already showed that the recipe can lose twice as many digits as necessary. This section explains why.

10.1.1 Floating-point numbers

A double-precision number is zero or a number $\pm m \times 2^{e-52}$, where m is an integer with $2^{52} \leq m < 2^{53}$ and e is an integer in a fixed range ($-1022 \leq e \leq 1023$). Between 2^e and 2^{e+1} the doubles are therefore equally spaced, 2^{e-52} apart. Just above 1 the spacing is $\epsilon_M = 2^{-52} \approx 2.22 \times 10^{-16}$, the **machine epsilon**. Rounding a real number x in the range of the doubles to the nearest double gives a number $\text{fl}(x)$ with

$$\text{fl}(x) = x(1 + \delta), \quad |\delta| \leq u = 2^{-53} \approx 1.11 \times 10^{-16}.$$

The **unit roundoff** $u = \epsilon_M/2$ is half the gap, because the nearest double is at most half a gap away. Rounding errors are *relative*: a double represents large and small numbers with the same number of significant digits, about sixteen.

The IEEE 754 standard, which essentially all current hardware follows, requires the basic operations to be correctly rounded. The computed result of $x + y$ is the exact sum, rounded. That is the only property of the hardware that the analysis below uses.

Definition 10.1.1 — Standard model of floating-point arithmetic

For floating-point numbers x, y and $\circ \in \{+, -, \times, /\}$, the computed result satisfies

$$\text{fl}(x \circ y) = (x \circ y)(1 + \delta), \quad |\delta| \leq u,$$

and $\text{fl}(\sqrt{x}) = \sqrt{x}(1 + \delta)$ with $|\delta| \leq u$, provided no result overflows or underflows.

Two consequences are worth seeing at once. The number $1 + u$ lies exactly halfway between the doubles 1 and $1 + \epsilon_M$, and the tie is broken towards 1, so adding u to 1 changes nothing. And most decimal fractions are not doubles, so familiar identities fail at the level of u :

```
import numpy as np

eps = np.finfo(float).eps          # spacing of the doubles just above 1: 2^-52
u = eps / 2                        # unit roundoff: 2^-53
print("eps =", eps, " u =", u)
print("1 + u == 1:", 1.0 + u == 1.0, " 1 + eps == 1:", 1.0 + eps == 1.0)
print("0.1 + 0.2 == 0.3:", 0.1 + 0.2 == 0.3)
```

[Run this code online](#)

Trouble comes from accumulation and above all from subtraction. The difference of nearly equal numbers is computed exactly but exposes the errors they already carried: if a and b agree in

ten digits, a rounding error in either becomes a large relative error in $a - b$. This **cancellation** is behind every numerical difficulty in this chapter.

10.1.2 Accumulating rounding errors

Products of many factors $1 + \delta_i$ appear whenever errors accumulate. The following lemma packages them.

Lemma 10.1.1

Let $|\delta_i| \leq u$ and $\rho_i \in \{1, -1\}$ for $i = 1, \dots, k$, and suppose $ku < 1$. Then

$$\prod_{i=1}^k (1 + \delta_i)^{\rho_i} = 1 + \theta_k, \quad |\theta_k| \leq \gamma_k = \frac{ku}{1 - ku}. \quad (10.1.1)$$

Proof. Each factor lies between $1 - u$ and $1/(1 - u)$: for $\rho_i = 1$ because $1 + u \leq 1/(1 - u)$, and for $\rho_i = -1$ because $1/(1 + \delta_i) \geq 1/(1 + u) \geq 1 - u$. So the product lies between $(1 - u)^k$ and $(1 - u)^{-k}$. By Bernoulli's inequality $(1 - u)^k \geq 1 - ku$, which gives the lower end $1 - ku \geq 1 - \gamma_k$, and also the upper end $(1 - u)^{-k} \leq 1/(1 - ku) = 1 + \gamma_k$. ■

The constant γ_k is essentially ku : for $k = 10^6$ it is about 1.1×10^{-10} . This is a worst case; in practice errors of different signs partly cancel and grow more like $\sqrt{k}u$.

The basic step of every least squares algorithm is an inner product.

Proposition 10.1.2 — Rounding error in an inner product

Let $\mathbf{x}, \mathbf{y} \in \mathbb{R}^n$ have floating-point entries and $nu < 1$. The inner product computed by recursive summation, $s_1 = \text{fl}(x_1 y_1)$ and $s_k = \text{fl}(s_{k-1} + \text{fl}(x_k y_k))$, satisfies

$$s_n = \sum_{i=1}^n x_i y_i (1 + \theta_i), \quad |\theta_i| \leq \gamma_n.$$

Consequently $|s_n - \mathbf{x}^\top \mathbf{y}| \leq \gamma_n \sum_i |x_i y_i|$, and $s_n = (\mathbf{x} + \Delta \mathbf{x})^\top \mathbf{y}$ for a vector $\Delta \mathbf{x}$ with $|\Delta x_i| \leq \gamma_n |x_i|$.

Proof. By the standard model, $\text{fl}(x_k y_k) = x_k y_k (1 + \mu_k)$ and $s_k = (s_{k-1} + x_k y_k (1 + \mu_k))(1 + \sigma_k)$ for $k \geq 2$, with all $|\mu_k|, |\sigma_k| \leq u$. Unrolling the recursion,

$$s_n = \sum_{i=1}^n x_i y_i (1 + \mu_i) \prod_{k=\max(i,2)}^n (1 + \sigma_k).$$

The i th term carries at most n factors of the form $1 + \delta$, so by [Lemma 10.1.1](#) it equals $x_i y_i (1 + \theta_i)$ with $|\theta_i| \leq \gamma_n$ (since $\gamma_k \leq \gamma_n$ for $k \leq n$). The two consequences follow by taking $\Delta x_i = \theta_i x_i$.

The first conclusion is a *forward* bound, which can be large relative to a result whose terms cancel. The second is a *backward* statement: the computed number is the exact inner product of slightly perturbed data. Over 200 random inner products of length up to 400, checked in rational arithmetic, the largest ratio of error to bound is 0.149.

```
from fractions import Fraction

def gamma(k, u=2.0 ** -53):
    return k * u / (1 - k * u)

def dot_recursive(x, y):
    """Inner product by recursive summation, one rounding per operation."""
    s = 0.0
    for a, b in zip(x, y):
        s = s + a * b
    return s

rng = np.random.default_rng(101)
worst = 0.0
for trial in range(200):
    n = int(rng.integers(2, 400))
    x = rng.normal(size=n) * 10.0 ** rng.integers(-3, 4, size=n)
    y = rng.normal(size=n)
    exact = sum(Fraction(a) * Fraction(b) for a, b in zip(x, y))      # exact
    rational value
    error = abs(Fraction(dot_recursive(x, y)) - exact)
    bound = Fraction(gamma(n)) * sum(abs(Fraction(a) * Fraction(b)) for a, b in zip(
        x, y))
    worst = max(worst, float(error / bound))
print(f"largest error / bound over 200 inner products: {worst:.3f}")
```

[Run this code online](#)

10.1.3 Conditioning and stability

A computation turns data d into a result $f(d)$. The computed result can be inaccurate because the *problem* is sensitive, or because the *algorithm* adds more error than the data justify. The first is measured by a **condition number**, the largest factor by which a small relative change in d can be magnified in $f(d)$; the second by the following standard.

Definition 10.1.2 — Backward stability

An algorithm that computes $f(d)$ is **backward stable** if, for every input d , its computed output $\hat{f}(d)$ is the exact result for nearby data: $\hat{f}(d) = f(d + \Delta d)$ with $\|\Delta d\| \leq c u \|d\|$, where c is a modest constant depending only on the dimensions of the problem.

Proposition 10.1.2 says that recursive summation is a backward stable way to compute inner products, with $c = \gamma_n/u \approx n$. Backward stability is the most one can ask, because storing the data as doubles already perturbs them by u . The two notions combine, to first order in u , as

$$\text{relative forward error} \lesssim \text{condition number} \times \text{backward error}. \quad (10.1.2)$$

For least squares the condition number, which involves $\kappa(\mathbf{X})$ and sometimes $\kappa(\mathbf{X})^2$, is the subject of [Section 10.5](#). Sections 10.2 to 10.4 ask whether algorithms are backward stable *for the least squares problem*, with perturbations of $\mathbf{X}$ and $\mathbf{y}$.

10.1.4 What forming the cross-product matrix loses

The normal-equations recipe is not backward stable for least squares, and the damage is done in its very first step. Here is what rounding does to $\mathbf{X}^\top \mathbf{X}$.

Proposition 10.1.3 — Rounding in the cross-product matrix

Let $\mathbf{X}$ be $n \times p$ with full column rank and let $\hat{\mathbf{A}}$ be $\mathbf{X}^\top \mathbf{X}$ computed entry by entry as in [Proposition 10.1.2](#). Then $\hat{\mathbf{A}} = \mathbf{X}^\top \mathbf{X} + \Delta \mathbf{A}$ with $|\Delta \mathbf{A}| \leq \gamma_n |\mathbf{X}|^\top |\mathbf{X}|$ entrywise, and

$$\|\Delta \mathbf{A}\|_2 \leq p \gamma_n \|\mathbf{X}\|_2^2.$$

In particular $\hat{\mathbf{A}}$ is positive definite whenever $p \gamma_n \kappa(\mathbf{X})^2 < 1$.

Proof. The entrywise bound is [Proposition 10.1.2](#) applied to each pair of columns. The spectral norm is at most the Frobenius norm, the Frobenius norm is monotone in the absolute values of the entries and submultiplicative, and $\|\mathbf{X}\|_F^2 = \sum_i \sigma_i^2 \leq p \sigma_1^2$ by [Proposition 1.8.2\(a\)](#). Hence

$$\|\Delta \mathbf{A}\|_2 \leq \gamma_n \|\mathbf{X}\|^\top \|\mathbf{X}\|_F \leq \gamma_n \|\mathbf{X}\|_F^2 \leq p \gamma_n \sigma_1^2.$$

For a unit vector $\mathbf{v}$, $\mathbf{v}^\top \hat{\mathbf{A}} \mathbf{v} = \|\mathbf{X} \mathbf{v}\|^2 + \mathbf{v}^\top \Delta \mathbf{A} \mathbf{v} \geq \sigma_p^2 - p \gamma_n \sigma_1^2$, which is positive when $p \gamma_n \sigma_1^2 / \sigma_p^2 < 1$. ■

The information about $\mathbf{X}$ that sits in its smallest singular value σ_p appears in $\mathbf{X}^\top \mathbf{X}$ as an eigenvalue σ_p^2 . Rounding perturbs $\mathbf{X}^\top \mathbf{X}$ by an amount of order $u \sigma_1^2$. Once σ_p^2 is below that level, the computed cross-product matrix may not determine the direction $\mathbf{v}_p$ at all. Even rounding the *exact* $\mathbf{X}^\top \mathbf{X}$ to doubles does this much damage. The loss becomes total around $\kappa(\mathbf{X}) \approx u^{-1/2} = 9.5 \times 10^7$, and it is already serious well before that: the normal equations are then a problem with condition number $\kappa(\mathbf{X}^\top \mathbf{X}) = \kappa(\mathbf{X})^2$ ([Eq. 1.8.2](#)), and [Eq. 10.1.2](#) predicts relative errors of order $\kappa(\mathbf{X})^2 u$ in $\hat{\boldsymbol{\beta}}$ whatever method is then used to solve them.

Example. (A cross-product matrix that rounds to a singular one)

Take $\delta = 10^{-9}$ and

$$\mathbf{X} = \begin{pmatrix} 1 & 1 \\ \delta & 0 \\ 0 & \delta \end{pmatrix}, \quad \mathbf{X}^\top \mathbf{X} = \begin{pmatrix} 1 + \delta^2 & 1 \\ 1 & 1 + \delta^2 \end{pmatrix}.$$

The columns of $\mathbf{X}$ are independent, and $\kappa(\mathbf{X}) = \sqrt{2 + \delta^2} / \delta = 1.41 \times 10^9$. But $\delta^2 = 10^{-18}$

is smaller than u , so $1 + \delta^2$ rounds to 1 and the computed $\mathbf{X}^\top \mathbf{X}$ is the singular matrix of ones. The Cholesky factorization of Section 10.2 fails, and no method applied to the computed normal equations can recover the solution. For $\mathbf{y} = \mathbf{X}(1, 1)^\top$, the exact least squares estimate is $(1, 1)^\top$ with zero residual. QR, which never forms $\mathbf{X}^\top \mathbf{X}$, returns it with relative error 3.9×10^{-16} .

```
delta = 1e-9
X = np.array([[1.0, 1.0],
              [delta, 0.0],
              [0.0, delta]])
y = X @ np.array([1.0, 1.0])          # exact data: the solution is b = (1, 1),
                                     # residual 0

A = X.T @ X                          # 1 + delta^2 rounds to 1
print("computed X'X =\n", A)
print("cond(X) =", np.linalg.cond(X))
try:
    np.linalg.cholesky(A)
except np.linalg.LinAlgError as err:
    print("Cholesky of the computed X'X fails:", err)

Q, R = np.linalg.qr(X)
b_qr = np.linalg.solve(R, Q.T @ y)
print("QR solution:", b_qr)
```

[Run this code online](#)

10.1.5 Why not the inverse either

Even when $\mathbf{X}^\top \mathbf{X}$ is safely positive definite, the inverse costs p^3 flops against $p^3/3$ for a Cholesky factorization plus $2p^2$ for two triangular solves, and it is less accurate. Statistics needs only the diagonal of $\text{Cov}(\hat{\boldsymbol{\beta}}) = \sigma^2(\mathbf{X}^\top \mathbf{X})^{-1}$ (Theorem 5.5.1): if $\mathbf{X}^\top \mathbf{X} = \mathbf{R}^\top \mathbf{R}$, its j th entry is σ^2 times the squared length of row j of $\mathbf{R}^{-1}$.

10.1.6 Counting the work

Cost is traditionally counted in floating-point operations (**flops**), each addition, subtraction, multiplication and division counting one. Flop counts ignore memory traffic and parallelism, so they are only a first guide.

Task	Leading flop count	Where
Form $\mathbf{X}^\top \mathbf{X}$ and $\mathbf{X}^\top \mathbf{y}$ (using symmetry)	np^2	this section
Cholesky factorization of $\mathbf{X}^\top \mathbf{X}$	$p^3/3$	Section 10.2
Two triangular solves	$2p^2$	Section 10.2
Householder QR of $\mathbf{X}$, with $\mathbf{Q}^\top \mathbf{y}$	$2np^2 - 2p^3/3$	Section 10.3
Givens QR of $\mathbf{X}$	$3np^2 - p^3$	Section 10.3

Task	Leading flop count	Where
Modified Gram–Schmidt	$2np^2$	Section 10.3
Singular values and $\mathbf{V}$, QR first (R-SVD)	$2np^2 + 11p^3$	Section 10.4

When $n \gg p$, as usual in regression, the normal equations cost about np^2 and Householder QR about $2np^2$: the accurate method is at most twice as expensive. And the one-pass, $O(p^2)$ -memory accumulation of cross-products has a QR counterpart in row-wise Givens QR ([Section 10.3](#)).

Idea. Forming $\mathbf{X}^\top \mathbf{X}$ squares the condition number before any equation is solved, and inverting it adds cost and error. Factor $\mathbf{X}$ itself, by an orthogonal transformation, and read everything off the factors.

10.1.7 Exercises

10.1.7.1 A. Check your understanding

Exercise A1.

Show that the doubles in $[1, 2)$ are exactly the numbers $1 + k 2^{-52}$, $k = 0, 1, \dots, 2^{52} - 1$. What is the smallest double greater than 1? What is the largest integer N such that every integer from 0 to N is a double?

Exercise A2.

For $x = 10^{-10}$, the computer evaluates $(1 + x) - 1$. Use the standard model to show that the result is $x + \delta(1 + x)$ for some $|\delta| \leq u$, where the subtraction itself is exact. Estimate the relative error, and check it numerically. Which of the two operations is responsible?

Solution. The addition gives $\text{fl}(1 + x) = (1 + x)(1 + \delta)$. This number is within a factor of two of 1, and the difference of two doubles within a factor of two of each other is itself a double, so the subtraction is exact: $\text{fl}(1 + x) - 1 = x + \delta(1 + x)$. The relative error is $|\delta|(1 + x)/x \approx u/x \approx 10^{-6}$. All of it comes from rounding $1 + x$, which kept only about six significant digits of x . The subtraction then exposed that loss.

10.1.7.2 B. Practice

Exercise B1.

Show that recursive summation of $x_1, \dots, x_n$ returns $\sum_i x_i(1 + \theta_i)$ with $|\theta_i| \leq \gamma_{n-1}$, and that pairwise summation returns such a sum with $|\theta_i| \leq \gamma_{\lceil \log_2 n \rceil}$.

Solution. With $s_1 = x_1$ and $s_k = (s_{k-1} + x_k)(1 + \sigma_k)$, $s_n = \sum_i x_i \prod_{k=\max(i,2)}^n (1 + \sigma_k)$. Term i has at most $n - 1$ factors, so [Lemma 10.1.1](#) gives $|\theta_i| \leq \gamma_{n-1}$. In pairwise summation each x_i passes through one addition per level of the tree, and a balanced tree

over n leaves has $\lceil \log_2 n \rceil$ levels. The same unrolling gives at most $\lceil \log_2 n \rceil$ factors per term.

Exercise B2.

Let $\mathbf{A}$ be $m \times n$ and $\mathbf{b} \in \mathbb{R}^n$. Show that the computed product $\text{fl}(\mathbf{A}\mathbf{b})$, each entry an inner product, equals $(\mathbf{A} + \Delta\mathbf{A})\mathbf{b}$ with $|\Delta\mathbf{A}| \leq \gamma_n |\mathbf{A}|$. Deduce $\|\text{fl}(\mathbf{A}\mathbf{b}) - \mathbf{A}\mathbf{b}\| \leq \gamma_n \sqrt{n} \|\mathbf{A}\|_2 \|\mathbf{b}\|$.

10.1.7.3 C. Going deeper

Exercise C1.

For $0 < \delta \leq 2^{-27}$ let $\mathbf{X}_\delta$ be the matrix of [Example \(A cross-product matrix that rounds to a singular one\)](#), and let $\mathbf{y} = \mathbf{X}_\delta(3, 1)^\top$ and $\mathbf{y}' = \mathbf{X}_\delta(1, 3)^\top$. Show that the computed cross-product matrices of $[\mathbf{X}_\delta, \mathbf{y}]$ and $[\mathbf{X}_\delta, \mathbf{y}']$ are identical (and do not depend on δ), although the exact least squares estimates are $(3, 1)^\top$ and $(1, 3)^\top$. Conclude that no algorithm that starts from the computed cross-product matrix can solve these problems. Is anything special about 2^{-27} ?